

# Set data type

---

A collection of unique elements enclosed within curly brackets { } separated by commas is called a Set. So we can say set is a collection of unique values.

1. Collection of unique elements enclosed within {} is called a set.
2. Duplicate values are NOT allowed.
3. Unordered collection (no fixed order).
4. Indexing not supported.
5. Slicing not supported.
6. Different data types allowed.
7. Mutable in nature (elements can be added or removed).

```
s1 = {10, 20, 30, 40}
s2 = {"Python", "Java", "C++"}
s3 = {1, "Rahul", 3.5, True}
```

```
print(s1)
print(s2)
print(s3)
```

```
# Output (order may vary)
{40, 10, 20, 30}
{'Python', 'Java', 'C++'}
{1, 'Rahul', 3.5, True}
```

## Duplicate Values in Set:-

```
s = {10, 20, 20, 30, 40, 10}
print(s)
```

```
# Output
{40, 10, 20, 30}
```

## Creating Empty Set:-

```
s = set()
print(type(s))
```

```
# Output
<class 'set'>
```

## Python In-built functions for Set:-

| Step | Syntax  | Explanation                     |
|------|---------|---------------------------------|
| 1    | len()   | Number of elements in set       |
| 2    | type()  | Check datatype                  |
| 3    | id()    | Check memory address            |
| 4    | set()   | Create empty set / type casting |
| 5    | print() | Print output                    |

```
s = {10, 20, 30}
```

```
print(len(s))  
print(type(s))  
print(id(s))
```

```
# Output
```

```
3  
<class 'set'>  
135446907837040
```

## Set Methods:-

### Methods that required single set:

1. add()
2. update()
3. remove()
4. discard()
5. pop()
6. clear()
7. copy()

#### 1. add():-- Adds a single element

```
s = {10, 20}  
s.add(30)  
print(s)
```

```
# O/P
```

```
{10, 20, 30}
```

#### 2. update():-- Adds multiple elements

```
s = {10, 20}
s.update([30, 40, 50])
print(s)
```

*# O/P*

```
{40, 10, 50, 20, 30}
```

### 3. remove():-- Removes element

```
s = {10, 20, 30}
s.remove(20)
print(s)
```

*# O/P*

```
{10, 30}
```

### 4. discard():-- Removes element (no error)

```
s = {10, 20, 30}

s.discard(40)
```

```
print(s)
```

*# O/P*

```
{10, 20, 30}
```

### 5. pop():-- Removes random element

```
s = {10, 20, 30}
s.pop()
print(s)
```

*# O/P (random)*

```
{20, 30}
```

### 6. clear():-- Removes all elements

```
s = {10, 20, 30}
s.clear()
print(s)
```

*# O/P*

```
set()
```

### 7. copy():-- Create new object with same elements of set

```
s1 = {10, 20, 30}
s2 = s1.copy()
print(s2)
```

*# O/P*

```
{10, 20, 30}
```

### **Methods that required more than one set:**

1. union()
2. intersection()
3. difference()
4. symmetric\_difference()
5. intersection\_update()
6. difference\_update()
7. symmetric\_difference\_update()
8. issubset()
9. issuperset()
10. isdisjoint()

### 1. Union



A = {1,2,3,4,5,6,7}  
B = {5,6,7,8,9,10}  
`print(A.union(B))`  
O/P:--  
{1,2,3,4,5,6,7,8,9,10}

### 2 Intersection



A = {1,2,3,4,5,6,7}  
B = {5,6,7,8,9,10}  
`print(A.intersection(B))`  
O/P:--  
{5,6,7}

### 3. Difference



A = {1,2,3,4,5,6,7}  
B = {5,6,7,8,9,10}  
`print(A.difference(B))`  
O/P:--  
{1,2,3,4}

### 4. Symmetric\_difference



A = {1,2,3,4,5,6,7}  
B = {5,6,7,8,9,10}  
`print(A.symmetric_difference)`  
O/P:-- {1,2,3,4,8,9,10}

### 5. Intersection\_update



```
A = {1,2,3,4,5,6}
B = {5,6,7,8,9}
A.intersection_update(B)
print(A)
O/P:-- {5,6}
```

### 6. difference\_update



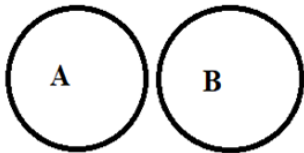
```
A = {1,2,3,4,5,6}
B = {5,6,7,8,9}
A.difference_update(B)
print(A)
O/P:--
{1,2,3,4}
```

### 7. symmetric\_difference\_update

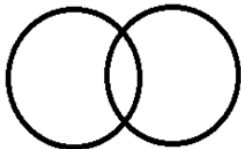


```
A = {1,2,3,4,5,6}
B = {5,6,7,8,9}
A.symmetric_difference_update(B)
print(A)
O/P:--
{1,2,3,4,7,8,9}
```

### 8. isdisjoint

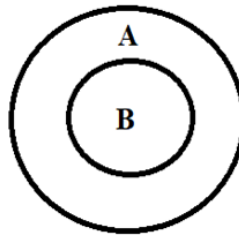


```
print(A.isdisjoint(B))
O/P:-
True
```

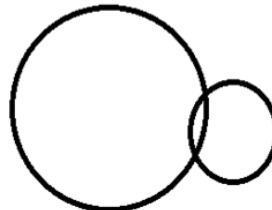


```
print(A.isdisjoint(B))
O/P:- False
```

### 9. issuperset

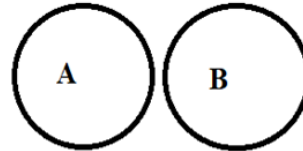


```
print(A.issuperset(B))
O/P:-- True
```

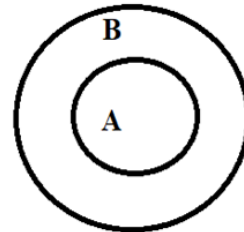


```
print(A.issuperset(B))
O/P:-- False
```

### 10. issubset



```
print(A.subset(B))
O/P:-- False
```



```
print(A.issubset(B))
O/P:-- True
```